

LEVEL 3 — HARD

Q1

recursive



Q2

functionsandpointers



2 / 2 saved

Write a recursive function to find the Nth Fibonacci number.

Sample Input:

Enter the value of N: 7

Sample Output:

7th Fibonacci number: 13

Test Cases:

1.5

2.10

3.15

4.20

Your Code

```
1 #include <iostream>
2 using namespace std;
3 int fibonacci(int n) {
4     if(n<=1) {
5         return n;
6     }
7     return fibonacci(n-1) + fibonacci(n-2);
8 }
9 int main() {
10     int n;
11     cout<<"Enter the value of N: "<<endl;
12     cin>>n;
13     cout<<n<<"th Fibonacci number: "<<fibonacci(n);
14     return 0;
15 }
```

Output

```
Enter the value of N:
15th Fibonacci number: 610
```

Q1

recursive



Q2

functions and pointers



2 / 2 saved

Write a function `int sumOfDiagonals(int *matrix, int n)` that calculates the sum of the diagonal elements of a square matrix using pointers.

Sample Input:

Enter the size of the matrix: 3

Enter elements of the matrix:

1 2 3

4 5 6

7 8 9

Sample Output:

Sum of diagonal elements is 15

Test Case 1: Enter number of rows: 3 Enter number of columns: 3 Enter elements: 1 2 3 4 5 6 7 8 9

Test Case 2: Enter number of rows: 2 Enter number of columns: 4 Enter elements: 1 2 3 4 5 6 7 8

Test Case 3: Enter number of rows: 4 Enter number of columns: 2 Enter elements: 1 2 3 4 5 6 7

Your Code

```
1 #include <iostream>
2 using namespace std;
3 int sumOfDiagonals(int *p, int n){
4     int sum=0;
5     for(int i=0;i<n;i++){
6         sum+=*(p+i*n+i);
7         sum+=*(p+i*n+(n-i-1));
8     }
9     if(n%2!=0)
10         sum-=*(p+(n/2)*n+(n/2));
11     return sum;
12 }
```

Output

```
Enter matrix size:
Enter matrix elements:
Sum of the diagonals: 25
```